

UNIVERSAL BUSINESS CORE

Hoja de ruta arquitectónica para una plataforma modular de reservas, pedidos, delivery, atención automatizada y canales digitales

Objetivo	Construir un backend universal y desacoplado, reutilizable en distintos tipos de negocio.
Primer vertical	Pica pollo dominicano: pedidos, recogida, reparto, reservas y WhatsApp.
Principio rector	El vertical configura y extiende el core; no introduce reglas sectoriales dentro del núcleo.
Stack inicial	Python, FastAPI, PostgreSQL, SQLAlchemy, Alembic, pytest; React/React Native como consumidores posteriores.

1. Visión y alcance

La plataforma se diseñará como un motor empresarial horizontal. El sistema debe representar capacidades comunes - catálogo, clientes, pedidos, reservas, recursos, disponibilidad, fulfillment, comunicaciones y agentes - sin conocer la tipología concreta del negocio.

El pica pollo será el primer caso de uso completo y, posteriormente, un segundo vertical deliberadamente distinto (por ejemplo, peluquería) servirá como prueba de desacoplamiento.

- Core agnóstico: ninguna entidad central debe llamarse Restaurant, PicaPollo, Salon o Clinic.
- Configuración antes que bifurcación: activar capacidades mediante configuración y feature flags.
- Ports & adapters: WhatsApp, push, pagos, mapas o proveedores LLM serán adaptadores reemplazables.
- Reglas deterministas: precios, disponibilidad, reservas, stock y estados se resuelven en servicios de dominio, nunca por el LLM.
- Multi-tenant desde el diseño: un mismo despliegue puede alojar varios negocios y ubicaciones con aislamiento lógico.

REGLA DE ORO	Si una necesidad sólo existe para el pica pollo, primero se busca su abstracción empresarial. Si no existe una abstracción sólida, se implementa como extensión del vertical y no como regla del core.
---------------------	--

Criterios de éxito arquitectónico

- Añadir un nuevo vertical sin modificar masivamente el dominio existente.
- Permitir web, app móvil, WhatsApp u otros canales sobre los mismos casos de uso.
- Mantener trazabilidad de cambios de estado y operaciones sensibles.
- Poder sustituir un proveedor externo sin reescribir reglas de negocio.
- Probar el dominio sin FastAPI, PostgreSQL, WhatsApp ni un modelo de IA en ejecución.

2. Arquitectura objetivo

Se recomienda un monolito modular inicialmente. Proporciona límites claros entre dominios sin introducir el coste operacional de microservicios prematuramente.

REST API Webhooks Admin API	Commands Queries Use cases Policies	Business Catalog Orders Reservations Availability Fulfillment	PostgreSQL WhatsApp Push Payments LLM

Módulos del Universal Core

Módulo	Responsabilidad
Identity & Access	usuarios, roles, permisos, API clients
Business	tenant, negocio, ubicaciones, horarios y configuración
Customers	clientes, contactos, preferencias y consentimiento
Catalog	productos, servicios, categorías, variantes, modificadores
Resources	recursos reservables: mesa, profesional, sala, pista, equipo
Availability	calendarios, capacidad, ventanas, bloqueos y reglas
Reservations	reservas/citas y ciclo de vida
Orders	carrito, líneas, precios, estados y totales
Fulfillment	on-site, pickup, delivery, shipping, digital
Notifications	eventos y mensajes salientes
Conversations	sesiones y mensajes multicanal
Agent Tools	herramientas seguras para consultar/ejecutar casos de uso
Audit & Events	trazabilidad y eventos de dominio
Integrations	adaptadores externos y webhooks

3. Modelo de dominio inicial

El modelo debe ser suficientemente general para representar venta de productos, prestación de servicios y reserva de recursos sin convertirlo en un metamodelo abstracto imposible de mantener.

Jerarquía de tenancy

Tenant → Business → Location. Las entidades operativas incluirán tenant_id, business_id y, cuando proceda, location_id. El aislamiento deberá validarse tanto en repositorios como en tests.

Entidades esenciales

Área	Entidades iniciales
Organización	Tenant, Business, Location, BusinessSettings
Identidad	User, Role, Membership
Cliente	Customer, ContactPoint, Address, Consent
Catálogo	CatalogItem, Product, Service, Category, Variant, ModifierGroup, ModifierOption
Recursos	Resource, ResourceType, Schedule, AvailabilityRule, Block
Reservas	Reservation, ReservationParticipant, ReservationStatus
Pedidos	Order, OrderItem, OrderAdjustment, OrderStatus
Cumplimiento	Fulfillment, FulfillmentType, DeliveryAddress, DeliveryZone
Comunicación	Conversation, Message, Channel, Notification
Trazabilidad	DomainEvent, AuditEntry, IdempotencyKey

Estados y contratos

- Los cambios de estado de Order y Reservation deben pasar por servicios de aplicación explícitos.
- Los importes monetarios se almacenarán con tipo decimal y moneda; nunca float.
- Las operaciones externas sensibles utilizarán idempotencia para evitar pedidos o reservas duplicados.
- Los eventos de dominio permitirán desacoplar notificaciones, auditoría y futuras integraciones.
- Los timestamps se almacenarán con zona horaria y las reglas de disponibilidad se evaluarán en la zona de la Location.

4. API, eventos y agente empresarial

La API será un contrato estable sobre los casos de uso. Los canales no escribirán directamente en tablas. Web, móvil y WhatsApp deberán recorrer las mismas reglas.

Superficie API orientativa

Dominio	Ejemplos
Business	GET /businesses/{id}, GET /locations/{id}
Catalog	GET /catalog, GET /catalog/items/{id}
Availability	POST /availability/search
Reservations	POST /reservations, PATCH /reservations/{id}/status
Orders	POST /orders, POST /orders/{id}/confirm, GET /orders/{id}
Fulfillment	POST /delivery/quote, POST /pickup/estimate
Customers	POST /customers, GET /customers/{id}
Webhooks	POST /webhooks/{provider}

Business Agent

El agente será una capa conversacional sobre herramientas controladas. El modelo interpreta intención y redacta respuestas; el backend conserva la autoridad sobre datos, precios, disponibilidad y transacciones.

Tool	Autoridad
get_business_info()	Información configurada del negocio
search_catalog()	Catálogo y disponibilidad real
check_availability()	Motor de disponibilidad
calculate_order()	Pricing del backend
create_order()	Order application service
create_reservation()	Reservation application service
get_order_status()	Estado persistido
cancel_order()	Políticas de cancelación
escalate_to_human()	Entrega de conversación a una persona

Guardrail fundamental: el agente no confirma una operación hasta recibir del servicio de aplicación el identificador y estado persistidos.

5. Hoja de ruta de implementación

La secuencia busca reducir incertidumbre arquitectónica antes de invertir en interfaces o integraciones externas.

FASE 0 · Contratos y arquitectura

Definir boundaries, ADRs, invariantes, estados, eventos, tenancy, estrategia de extensiones y criterios de aceptación.

Criterio de salida: Documento arquitectónico aprobado + skeleton del repositorio.

FASE 1 · Universal Core

Business, Location, Customer, Catalog, Resource, Availability, Reservation, Order y Fulfillment con tests unitarios.

Criterio de salida: Dominio ejecutable sin infraestructura externa.

FASE 2 · Persistencia y API

PostgreSQL, SQLAlchemy, Alembic, repositorios, FastAPI, validación, idempotencia y OpenAPI.

Criterio de salida: API funcional y suite de integración.

FASE 3 · Eventos y notificaciones

Outbox/event dispatcher, push, email/webhooks y auditoría.

Criterio de salida: Alertas fiables sin acoplarlas a Order/Reservation.

FASE 4 · Business Agent

Conversation layer, tools, políticas, contexto, escalado humano y pruebas de seguridad funcional.

Criterio de salida: Agente capaz de consultar y ejecutar operaciones autorizadas.

FASE 5 · Vertical Pica Pollo

Combos, modificadores, stock simple, pickup, delivery, zonas, reservas opcionales, seed y configuración.

Criterio de salida: Caso de uso completo sin contaminar el core.

FASE 6 · Web + panel + móvil

React para cliente/panel; React Native para propietario; notificaciones push.

Criterio de salida: MVP operativo extremo a extremo.

FASE 7 · WhatsApp

Webhook, conversación, plantillas/proveedor, agent tools y handoff.

Criterio de salida: Pedido/reserva conversacional con trazabilidad.

FASE 8 · Segundo vertical

Implementar peluquería u otro negocio de servicios sin rediseñar el core.

Criterio de salida: Prueba de generalidad arquitectónica.

FASE 9 · Hardening SaaS

Observabilidad, backups, rate limits, permisos finos, métricas, CI/CD, seguridad y operación.

Criterio de salida: Base comercializable y desplegable.

6. Vertical 1 - Pica pollo dominicano

Este vertical valida simultáneamente catálogo compuesto, pedidos, fulfillment y conversación. Debe implementarse como configuración, seeds y extensiones sectoriales.

Capacidades activadas

- Catálogo por categorías: pollo, combos, acompañamientos, bebidas y extras.
- Modifier groups para elegir acompañamiento, bebida, tamaño o extras.
- Pedidos para recoger, consumir en local y reparto a domicilio.
- Zonas de entrega, tarifa, pedido mínimo y estimación.
- Disponibilidad/stock simple para ocultar temporalmente productos o modificadores.
- Reservas de mesa sólo si el establecimiento las necesita.
- Notificación al propietario para nuevos pedidos y reservas.
- WhatsApp para FAQ, catálogo, toma guiada de pedido, estado y escalado.

Lo que NO debe entrar como concepto del core

Evitar en Core	Representación universal
PiezaDePollo	CatalogItem / Product
ComboPicaPollo	Bundle/Composition o configuración de producto
Yuca / Plátano	ModifierOption o Product
MesaRestaurante	Resource
RepartidorRestaurante	Fulfillment actor / extensión
PedidoPicaPollo	Order

Prueba de aceptación del vertical

Un cliente debe poder consultar el menú, construir un pedido válido, obtener el total calculado por backend, seleccionar pickup o delivery, confirmar, generar una alerta al propietario, consultar su estado y recibir actualizaciones; todo ello utilizando servicios universales del core.

7. Calidad, seguridad y observabilidad

La generalidad del sistema debe demostrarse con tests, no sólo con convenciones de nombres.

Pirámide de pruebas

Nivel	Objetivo
Unitarios de dominio	Invariantes, pricing, estados, disponibilidad, políticas
Application tests	Casos de uso con repositorios/adaptadores falsos
Integración	PostgreSQL, transacciones, aislamiento tenant, migraciones
API contract	OpenAPI, validación, errores, idempotencia
Adapter tests	WhatsApp/push/pagos mediante mocks o sandboxes
E2E	Flujos críticos del vertical
Architecture tests	Evitar imports prohibidos y dependencias inversas

Controles mínimos antes de producción

- Autenticación y RBAC para propietario, empleado, operador y administración.
- Aislamiento multi-tenant probado y filtros obligatorios por tenant.
- Secretos fuera del repositorio; rotación y configuración por entorno.
- Verificación de firmas de webhooks y protección frente a replay.
- Rate limiting en endpoints públicos y canales conversacionales.
- Registro de auditoría para cambios de pedido, reserva, precio y permisos.
- Backups y restauración probada de PostgreSQL.
- Minimización de datos personales, retención configurable y consentimiento cuando corresponda.
- Logs estructurados, métricas y correlation/request IDs.

8. Estructura propuesta del repositorio

Una estructura inicial explícita ayuda a preservar los límites cuando Codex u otros agentes comiencen a implementar funcionalidad.

```
src/  
  universal_business/  
    domain/  
      business/  
      customers/  
      catalog/  
      resources/  
      availability/  
      reservations/  
      orders/  
      fulfillment/  
      conversations/  
    application/  
      commands/  
      queries/  
      services/  
      ports/  
    infrastructure/  
      persistence/  
      notifications/  
      messaging/  
      llm/  
      payments/  
    api/  
      v1/  
      dependencies/  
      webhooks/  
    verticals/  
      pica_pollo/  
        config/  
        seeds/  
        extensions/  
    tests/  
      unit/  
      application/  
      integration/  
      architecture/  
    docs/  
      adr/  
      architecture/  
      api/
```

Decisiones que deben registrarse como ADR

- Monolito modular frente a microservicios.
- Estrategia multi-tenant y aislamiento.
- Modelo Product/Service/CatalogItem.
- Representación de recursos y disponibilidad.
- Eventos de dominio y patrón outbox.
- Idempotencia de pedidos/reservas.
- Modelo de extensiones verticales.

- Proveedor de autenticación y estrategia RBAC.
- Límites de responsabilidad del agente LLM.

9. Primer hito recomendado

No comenzar todavía por WhatsApp, React Native ni el diseño visual. El primer incremento debe demostrar que el dominio universal funciona por sí solo.

Entrega 0.1 - Architectural Baseline

- Repositorio Python inicial con pyproject.toml y configuración de calidad.
- Documento ARCHITECTURE.md y ADR-001 a ADR-005.
- Value objects básicos: IDs, Money, DateRange/TimeRange y estados.
- Tenant, Business, Location y Customer.
- Interfaces de repositorio (ports), sin dependencia de SQLAlchemy en domain.
- Esqueleto de Catalog, Resource, Reservation, Order y Fulfillment.
- Eventos de dominio base.
- Tests arquitectónicos que impidan dependencias indebidas.
- pytest + lint/type checking en CI.
- Sin frontend, WhatsApp, Firebase ni LLM en esta entrega.

GATE 0.1

La entrega se acepta sólo si el core puede importarse y probarse sin levantar FastAPI, una base de datos o servicios externos, y si no contiene nombres o reglas específicas del pica pollo.

Conclusión

La estrategia convierte el primer encargo en una plataforma reutilizable. El pica pollo aporta un caso de uso exigente y realista, pero la arquitectura conserva suficiente neutralidad para evolucionar hacia restaurantes, peluquerías, talleres, centros deportivos u otros negocios sin reconstruir el backend. El segundo vertical será la prueba definitiva de que la abstracción es correcta.